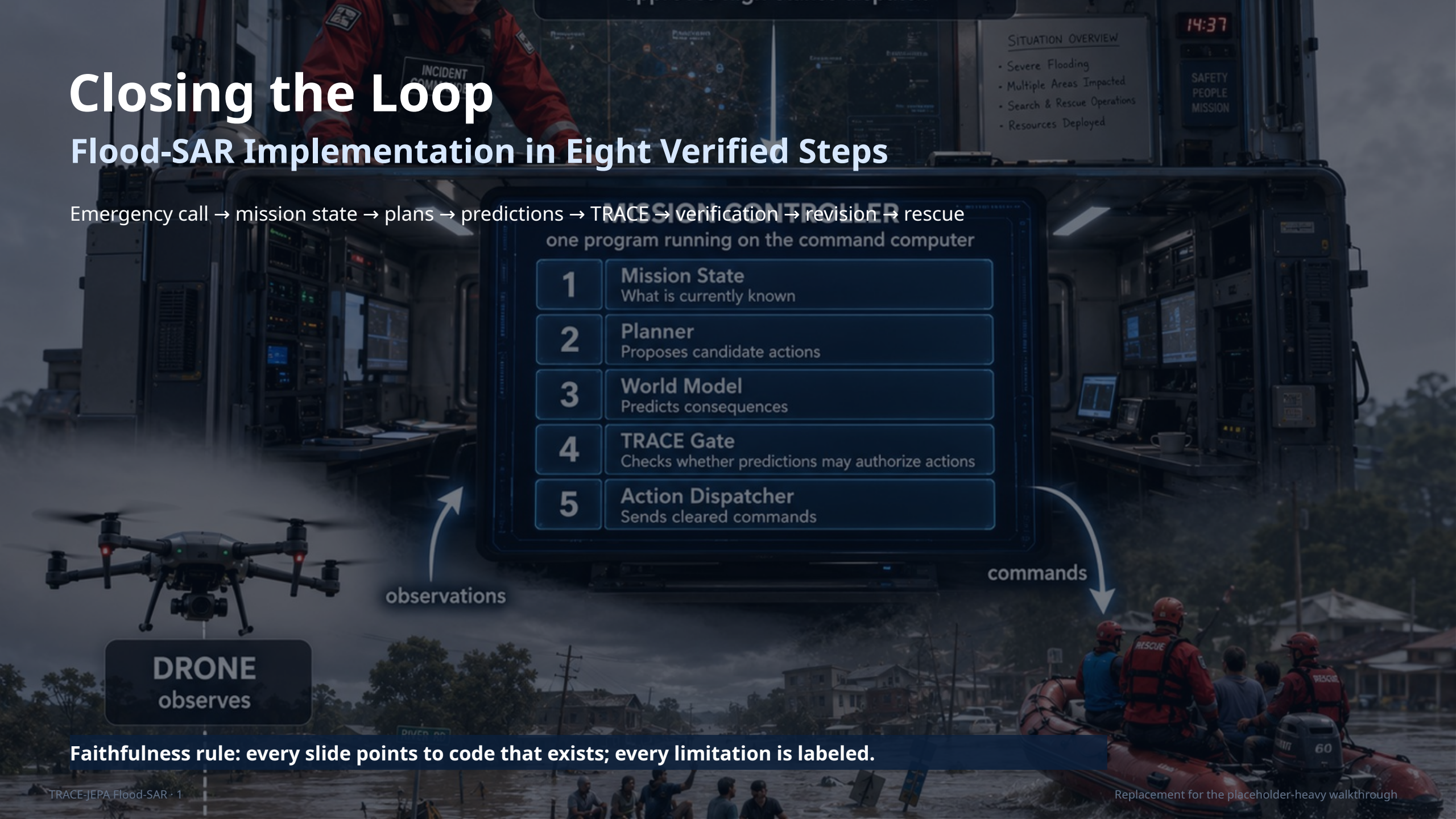
Closing the Loop

Flood-SAR Implementation in Eight Verified Steps

Emergency call → mission state → plans → predictions → TRACE → verification → revision → rescue



Faithfulness rule: every slide points to code that exists; every limitation is labeled.

Replacement for the placeholder-heavy walkthrough

What “eight steps” means

These are the eight operational responsibilities from call intake to a recorded rescue outcome.

1 Ground call

EmergencyCall object

2 Build mission state

Controller-visible observation

3 Propose plans

PlanCandidate list

4 Predict + claim

PlanPrediction + Claim

5 TRACE gate

TraceRecord + decision

6 Verify

Drone outcome

7 Revise + replan

Revision + candidate repair

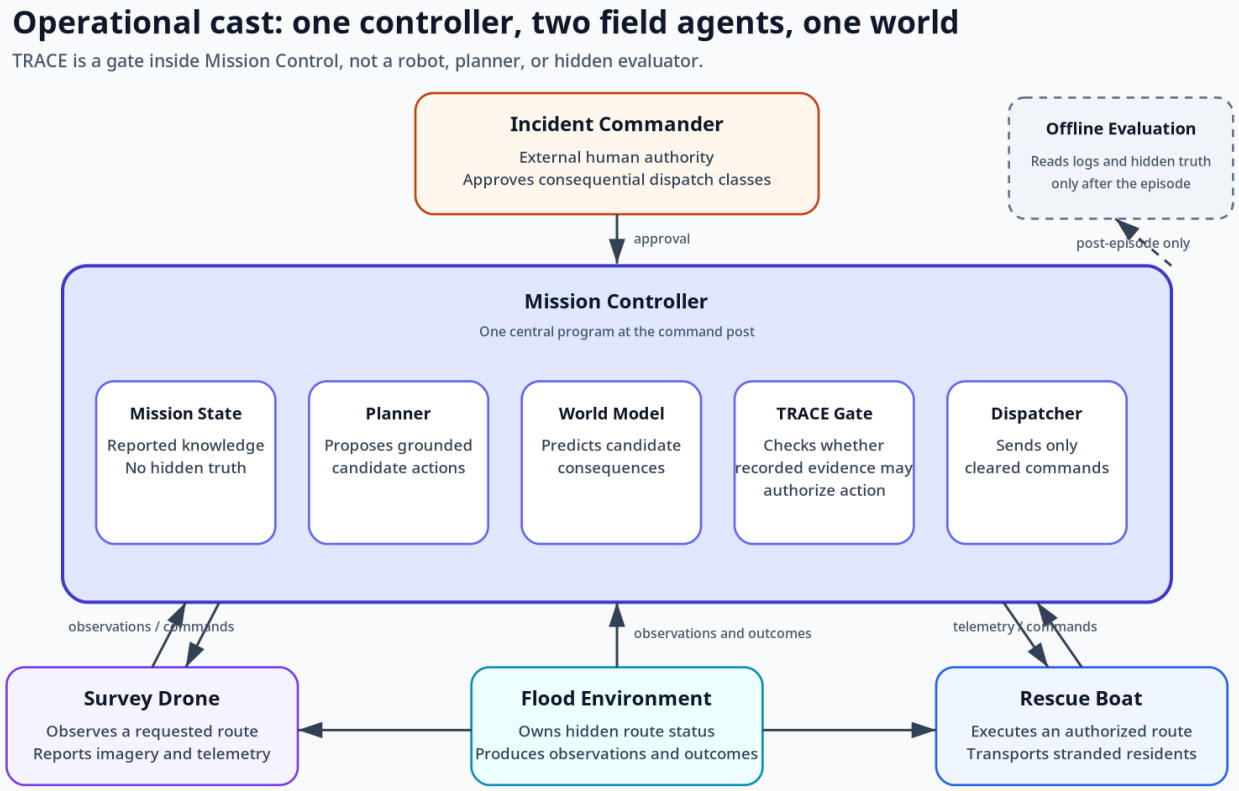
8 Rescue + persist

Commitment + outcome

Do not confuse these with TRACE’s eight general writer stages. The crosswalk later shows which writer operations this domain-specific implementation actually realizes.

One controller, two machines, one human, one world

The architecture is intentionally simple in the first implementation.



- Flood Environment owns hidden truth and produces observations/outcomes.
- Survey drone observes; rescue boat executes. Neither plans.
- Mission Controller is one Python process containing planner, predictor, TRACE gate, and dispatcher.
- Incident Commander is an external authority boundary.
- Offline evaluation scores logs after the episode and sends no commands.

Current honesty note: IncidentCommander.authorizes() is a deterministic teaching stub, not a real human approval interface.

Source: src/trace_jepa/controller.py

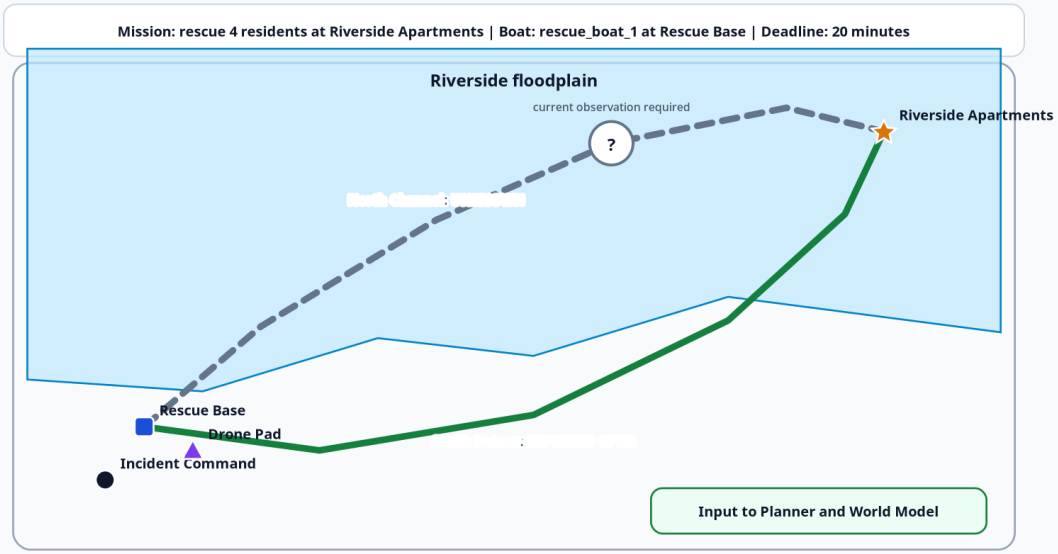
The knowledge boundary makes the episode scientifically valid

The controller knows what has been reported, not everything the simulator contains.

MISSION CONTROLLER KNOWLEDGE

Mission Controller knowledge at time zero

The North Channel is unverified. No obstruction is shown because none has been reported.

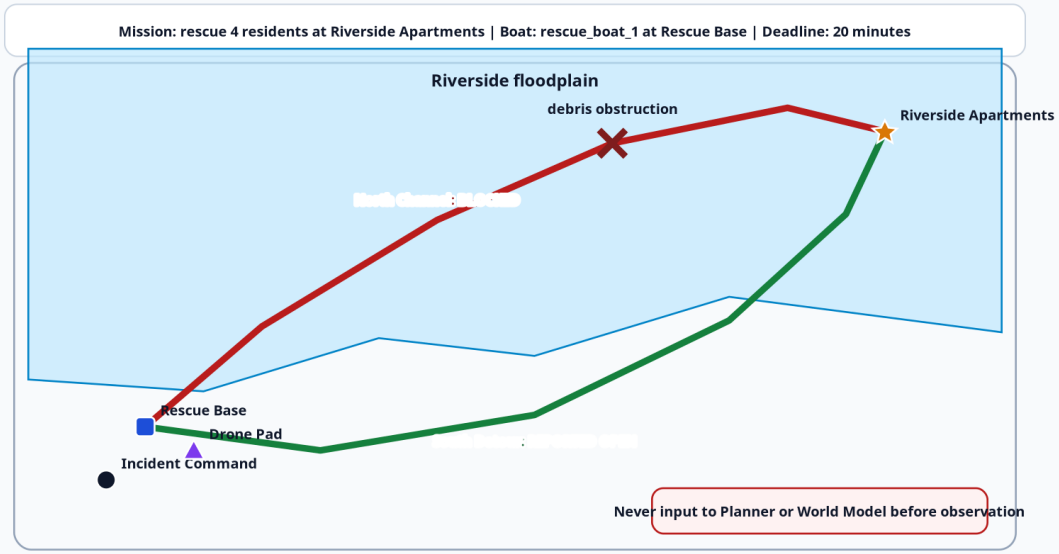


North Channel = unknown

SIMULATION GROUND TRUTH

Simulation ground truth: withheld from Mission Control

The environment contains a debris obstruction. This view is for teaching and post-episode scoring only.



North Channel = blocked

Hidden truth never enters planner/predictor/gate inputs

Run the implementation from a raw emergency call

One command produces the timeline, records, evidence, commitments, summary, and figures.

```
source "$HOME/miniforge3/etc/profile.d/conda.sh"
conda activate trace-jepa
cd /path/to/trace_jepa_flood_sar_starter
python -m pip install -e ".[dev]"
pytest

trace-jepa-call \
  --call-file examples/calls/riverside_call.txt \
  --output artifacts/runs/call_001
```

SIDEFILE:RUN_EIGHT_STEPS.md

Expected core test state: all core tests pass; the optional PyTorch/V-JEPA test may be skipped until the ML stack is installed.

Input

Emergency. Four residents are stranded at Riverside Apartments. Flood water is rising and the road is inaccessible.

Main outputs

emergency_call.json
timeline.txt
summary.json
records/trace.jsonl
evidence/*.json
commitments/*.jsonl
figures/*.svg

Entry point: trace_jepa.emergency_cli:main

Step 1: Accept and ground the emergency call

Convert raw caller language into an auditable EmergencyCall object without inventing missing fields.

STEP 1

Accept and ground the emergency call

Input

raw_text
optional --location
optional --people
optional deadline override

Processing

closed location alias registry
regex/number-word people count
scenario default deadline
explicit validation errors

Output

EmergencyCall
normalized_location_id
people_count
deadline_s
raw call preserved

- Ambiguous location → stop and request --location.
- Missing people count → stop and request --people.
- No geocoder is implemented yet.
- The call does not reveal route status.

```
Emergency. Four residents are stranded at Riverside Apartments.
Flood water is rising and the road is inaccessible.
```

SIDE FILE: step01_call_intake/S01_intake.py

SIDE FILE: step01_call_intake/S01_emergency_cli.py

Output artifact: emergency_call.json

Step 1: Code: parse the call

The slide shows the core logic; the complete source is in the labeled side file.

STEP 1

Code: parse the call

```python
def parse(
    self,
    raw_text: str,
    *,
    location: str | None = None,
    people: int | None = None,
    deadline_minutes: int | None = None,
    caller_name: str | None = None,
    caller_contact: str | None = None,
) -> EmergencyCall:
    if not raw_text.strip():
        raise EmergencyCallIntakeError("emergency call text cannot be empty")

    location_id, label = resolve_location(
        self.scenario,
        raw_text=raw_text,
        explicit_location=location,
    )
    people_count = parse_people_count(raw_text, explicit_people=people)

    default_deadline_s = int(self.scenario.get("mission", {}).get("deadline_s", 1200))
    deadline_s = (
        int(deadline_minutes) * 60
        if deadline_minutes is not None
        else default_deadline_s
    )
    if deadline_s < 60:
        raise EmergencyCallIntakeError("deadline must be at least one minute")

    return EmergencyCall(
        raw_text=raw_text.strip(),
        reported_location=location or label,
        normalized_location_id=location_id,
        people_count=people_count,
        deadline_s=deadline_s,
        caller_name=caller_name,
        caller_contact=caller_contact,
    )
```

- Location is normalized against scenario aliases.
- People count is parsed independently.
- Deadline is explicit and validated.
- EmergencyCall is frozen by the Pydantic contract.

Exit test

normalized_location_id = riverside_apartments
people_count = 4
deadline_s = 1200

Faithfulness boundary: this is deterministic text parsing, not speech recognition, NLP extraction, or geocoding.

SIDE FILE: step01_call_intake/S01_intake.py

Authoritative source: src/trace_jepa/intake.py

Step 2: Register the incident and build mission state

Create the controller-visible state while preserving hidden simulator truth.

STEP 2

Register the incident and build mission state

```
Mission Controller sees:
- destination: riverside_apartments
- people: 4
- deadline: 1200 s
- north_channel: unknown
- south_detour: open
- drone battery: 0.82
- boat capacity: 6
```

```
Simulator holds:
- north_channel: blocked
- south_detour: open
- stranded_people: 4
- rescued: false
```

Only observe() enters MissionController._assess_observation(). simulation_ground_truth() is retained for tests and post-episode analysis.

SIDE FILE: step02_mission_state/S02_flood_environment.py

SIDE FILE: step02_mission_state/S02_riverside_flood_v1.yaml

SIDE FILE: step02_mission_state/S02_visualize.py

SIDE FILE: step02_mission_state/S02_fusion_state.py

Exit test

Controller north report = unknown
Truth north status = blocked

Source: FloodEnvironment.register_emergency_call / observe

Step 2: Code: the information boundary

The key correctness property is that observe() masks the blocked route until verification.

STEP 2

Code: the information boundary

```python
def register_emergency_call(self, call: EmergencyCall) -> None:
    """Ground an emergency report into the active mission.

    The call may change the pickup location, number of residents, and
    deadline. It does not reveal hidden route truth.
    """

    if call.normalized_location_id not in self.scenario["locations"]:
        raise ValueError(
            f"unknown scenario location: {call.normalized_location_id}"
        )
    self.active_call = call
    self.scenario["mission"]["pickup_location"] = call.normalized_location_id
    self.scenario["mission"]["people_to_rescue"] = call.people_count
    self.scenario["mission"]["deadline_s"] = call.deadline_s

def observe(self) -> dict[str, Any]:
    """Return only information legitimately available to Mission Control."""

    routes: dict[str, dict[str, Any]] = {}
    for route_id, route in self.scenario["routes"].items():
        verified = route_id != "north_channel" or self.north_route_verified
        report = self._route_status(route_id) if verified else str(route["initial_report"])
        routes[route_id] = {
            "label": route["label"],
            "report": report,
            "verified": verified,
            "nominal_travel_s": float(route["nominal_travel_s"]),
            "waypoints": route["waypoints"],
        }

    assets = {
        asset_id: dict(spec)
        for asset_id, spec in self.scenario["assets"].items()
    }
    mission = dict(self.scenario["mission"])

    return {
        "scenario_id": self.scenario_id,
        "mission": {
            "origin_id": mission["origin"],
            "destination_id": mission["pickup_location"],
            "people_to_rescue": int(mission["people_to_rescue"]),
            "deadline_s": int(mission["deadline_s"]),
            "deadline_minutes": int(mission["deadline_s"]) // 60,
        },
```

- register_emergency_call changes mission parameters only.
- north_route_verified controls whether truth is exposed.
- initial_report is returned while unverified.
- simulation_ground_truth is a separate method.

The two SVG maps differ by one hidden fact. This is not cosmetic; it prevents ground-truth leakage.

SIDE FILE: step02_mission_state/S02_flood_environment.py

Test: tests/test_entity_model.py and test_emergency_call.py

Step 3: Generate grounded candidate actions

The planner says what is structurally possible; it does not predict or authorize.

STEP 3

Generate grounded candidate actions

North dispatch

rescue_boat_1
rescue_base → riverside_apartments
route=north_channel
irreversible
authority required

Drone verification

survey_drone_1
drone_pad → north_channel
purpose=resolve_route_access
reversible
no dispatch authority

South dispatch

rescue_boat_1
rescue_base → riverside_apartments
route=south_detour
irreversible
authority required

Every PlanCandidate contains an ActionInstance with actor, origin, destination, route, people count, deadline, reversibility, utility, and authority requirement.

SIDE FILE: step03_planning/S03_planner.py

SIDE FILE: step03_planning/S03_contracts_models.py

SIDE FILE: step03_planning/S03_flood_actions_v1.yaml

Exit test

Three initial candidates; no opaque action strings.

Step 3: Code: propose actions from observed route reports

The planner generates boat dispatches for non-blocked reports and verification actions for unknown reports.

STEP 3 **Code: propose actions from observed route reports**

```python
def propose(self, observation: dict) -> list[PlanCandidate]:
    mission = observation["mission"]
    candidates: list[PlanCandidate] = []

    for route_id, route in observation["routes"].items():
        report = route["report"]
        label = route["label"]

        if report != "blocked":
            action = ActionInstance(
                action_type="dispatch_rescue_boat",
                actor_id="rescue_boat_1",
                origin=mission["origin_id"],
                destination=mission["destination_id"],
                route_id=route_id,
                parameters={
                    "people_count": int(mission["people_to_rescue"]),
                    "deadline_s": int(mission["deadline_s"]),
                },
            )
            travel_s = float(route["nominal_travel_s"])
            utility = max(0.0, 1.0 - 0.35 * travel_s / float(mission["deadline_s"]))
            plan_id = "north-direct" if route_id == "north_channel" else "south-detour"
            candidates.append(
                PlanCandidate(
                    plan_id=plan_id,
                    name=(
                        f"Dispatch rescue_boat_1 from {mission['origin_id']} "
                        f"to {mission['destination_id']} via {label}"
                    ),
                    actions=(action,),
                    utility=utility,
                    reversible_first_action=False,
                    requires_authority=True,
                    metadata={
                        "route_id": route_id,
                        "route_report": report,
                        "mission_destination": mission["destination_id"],
                    },
                )
            )

        if report == "unknown":
            action = ActionInstance(
                action_type="verify_route",
                actor_id="survey_drone_1",
                origin="drone_pad",
                destination=route_id,
                route_id=route_id,
                parameters={"purpose": "resolve_route_access"},
            )
            candidates.append(
                PlanCandidate(
                    plan_id="verify-north" if route_id == "north_channel" else f"verify-{route_id}",
                    name=f"Send survey_drone_1 to verify {label}",
                    actions=(action,),
                    utility=0.90,
                    reversible_first_action=True,
                    requires_authority=False,
                    metadata={
                        "route_id": route_id,
                        "information_gathering": True,
                    },
                )
            )
```

- No prediction values appear in this file.
- Unknown is not treated as open.
- A blocked route is omitted on replanning.
- choose() ranks only CLEAR/QUALIFY candidates.

Current limitation: this is a small deterministic candidate generator, not an HTN/MRTA solver.

SIDE FILE: step03_planning/S03_planner.py

Authoritative source: src/trace_jepa/planning/planner.py

Step 4: Predict consequences and formulate claims

The current predictor is a transparent deterministic fixture, not JEPA and not an empirical result.

STEP 4

Predict consequences and formulate claims

Candidate	Success	Support	OOD	Uncertainty	Expected gate
North unknown	0.92	0.28	0.82	0.08	HOLD later
Verify north	0.98	0.97	0.04	0.04	CLEAR later
South open	0.82	0.93	0.08	0.14	CLEAR later

Claim generated for a boat plan: “north_channel is traversable and rescue_boat_1 can reach riverside_apartments before the deadline.”

SIDE FILE: step04_prediction_claims/S04_toy_predictor.py

SIDE FILE: step04_prediction_claims/S04_claim_probe.py

Exit test

Success, support, OOD, uncertainty, horizon, and assumptions remain separate fields.

Predictor version: toy-action-prefix-v2

Step 4: Code: the deliberate out-of-support teaching case

The predictor may report high success while also declaring low support and high OOD.

STEP 4

Code: the deliberate out-of-support teaching case

PREDICTOR FIXTURE

```
report = route["report"]
    nominal_travel_s = float(route["nominal_travel_s"])

    if report == "unknown":
      # Teaching case: a high numerical success estimate produced outside
      # the predictor's declared support. TRACE must hold the dispatch.
      return PlanPrediction(
        plan_id=plan.plan_id,
        success_probability=0.92,
        arrival_time_s=nominal_travel_s,
        hazard_score=0.12,
        resource_margin=0.35,
        model_support=0.28,
        out_of_distribution_score=0.82,
        uncertainty=0.08,
        rollout_horizon=6,
        assumptions=("map_is_current","debris_distribution_matches_training"),
      )
```

TYPED CLAIM PROBE

```
elif action.action_type == "dispatch_rescue_boat":
      text = (
        f"{action.route_id} is traversable and {action.actor_id} can reach "
        f"{action.destination} before the deadline."
      )
      grounding = {
        "actor_id": action.actor_id,
        "origin": action.origin,
        "destination": action.destination,
        "route_id": action.route_id,
        "people_count": action.parameters["people_count"],
        "deadline_s": action.parameters["deadline_s"],
        "horizon_steps": prediction.rollout_horizon,
        "action_id": action.action_id,
      }
    else:
      raise KeyError(action.action_type)

    return Claim(
      layer=ClaimLayer.PREDICTIVE,
      text=text,
      grounding=grounding,
      confidence=prediction.success_probability,
      confidence_semantics=(
        "Teaching proxy copied from plan success probability; not a "
        "separately calibrated claim probability."
      ),
    )
```

The code explicitly labels claim confidence as a teaching proxy copied from plan success probability. It is not presented as a separately calibrated claim probability.

SIDE FILE: step04_prediction_claims/S04_toy_predictor.py

SIDE FILE: step04_prediction_claims/S04_claim_probe.py

Faithful limitation: no learned world model is used yet

Step 5: Write TRACE records and apply the policy gate

Evidence is stored first; then a deterministic policy writes verdict, failed gates, missing evidence, repair, and consumer action.

STEP 5

Write TRACE records and apply the policy gate

North dispatch	**DEFER**	**HOLD**	model_support, out_of_distribution
Drive verification	**ACCEPT**	**CLEAR**	none
South dispatch	**ACCEPT**	**CLEAR**	none

SIDE FILE: step05_trace_gate/S05_policy.py

SIDE FILE: step05_trace_gate/S05_runtime.py

SIDE FILE: step05_trace_gate/S05_storage.py

Policy version: trace-flood-v1

Step 5: Code: hard gates cannot be averaged away

The 0.92 score is not read as permission when support and OOD checks fail.

STEP 5 **Code: hard gates cannot be averaged away**

```python
if evidence.model_support < self.config.min_model_support:
        failed.append("model_support")
        missing.append("evidence inside the declared model support")
    if evidence.out_of_distribution_score > self.config.max_ood_score:
        failed.append("out_of_distribution")
        missing.append("current observation or verification for the out-of-support region")
    if evidence.uncertainty > self.config.max_uncertainty:
        failed.append("uncertainty")
        missing.append("narrower calibrated uncertainty")
    if evidence.rollout_horizon > self.config.max_rollout_horizon:
        failed.append("rollout_horizon")
        missing.append("a shorter receding-horizon prefix")
    if evidence.observation_age_s > self.config.max_observation_age_s:
        failed.append("observation_freshness")
        missing.append("a fresh observation window")
    if claim.layer == ClaimLayer.CAUSAL and "state_sufficiency" not in evidence.assumptions:
        failed.append("causal_identification")
        missing.append("state-sufficiency or an explicitly model-conditional causal qualification")

    authority_required = action_name in self.config.require_authority_for
    if authority_required and not authority_present:
        return EvaluationResult(
            status=TraceStatus.DEFER,
            decision=CommitmentDecision.ESCALATE,
            failed_gates=tuple(failed),
            missing_items=tuple(missing + ["incident-command authorization"]),
            repair="Obtain authorization from the designated incident-command role.",
            reason="Technical evidence is not sufficient to cross the authority boundary.",
        )

    if failed:
        if reversible and self.config.allow_qualified_reversible_probe:
            return EvaluationResult(
                status=TraceStatus.QUALIFY,
                decision=CommitmentDecision.QUALIFY,
                failed_gates=tuple(failed),
                missing_items=tuple(missing),
                repair=(
                    repair_hint
                    or "Limit the action to evidence gathering; monitor and do not commit the rescue branch yet."
                ),
                reason="A bounded reversible probe may proceed to resolve the named uncertainty.",
            )
        return EvaluationResult(
            status=TraceStatus.DEFER,
            decision=CommitmentDecision.HOLD,
            failed_gates=tuple(failed),
            missing_items=tuple(missing),
            repair=(
                repair_hint
                or "Gather the named evidence or select a supported alternative plan."
            ),
            reason="One or more hard technical gates failed.",
        )

    return EvaluationResult(
        status=TraceStatus.ACCEPT,
        decision=CommitmentDecision.CLEAR,
        reason="All declared technical gates passed for this action class.",
```

- Failed technical gates + irreversible action → HOLD.
- Failed gates + reversible evidence probe → QUALIFY when enabled.
- Authority missing → ESCALATE.
- No failed gates → CLEAR.

North result

status = defer
decision = hold
missing = supported/current route evidence
repair = send drone

SIDE FILE: step05_trace_gate/S05_policy.py

Test: tests/test_policy.py

How this domain writer maps to TRACE’s eight writer stages

Faithfulness requires stating what is implemented and what is not.

0 Detect	Planner emits an explicit action-licensing claim candidate.	**No open-ended argument detector.**
1 Intake	Probe + controller fill grounding, evidence, provenance, assumptions.	**Implemented structurally.**
2 Type	ClaimLayer.PREDICTIVE.	**Implemented.**
3 Question	Support, OOD, uncertainty, horizon, freshness, contradiction, authority.	**Domain-specific question set.**
4 Test	PolicyEngine.evaluate runs deterministic checks.	**Implemented.**
5 Elicit	Failed record names drone verification as missing evidence/repair.	**No debate module.**
6 Settle	Policy maps tests to record status and consumer decision.	**No attack graph.**
7 Complete	Runtime writes record, gates, missing, repair, provenance, consumer action.	**Implemented.**

Conclusion: the current flood code is a conforming structured writer for predictive plan claims. It is not yet the full general-purpose eight-stage natural-language writer.

Do not claim debate or attack-graph implementation

Step 6: Select and execute the evidence-seeking action

The highest-utility admissible action is the drone verification, not the held north dispatch.

STEP 6

Select and execute the evidence-seeking action

Candidates

North HOLD
Verify CLEAR
South CLEAR

choose()

Rank only CLEAR/QUALIFY
Verify utility 0.90
South utility 0.685

commit()

Action + authorizing record ID/version

execute()

Drone battery 0.82→0.74
route_status=blocked

```
[06]ACTION SELECTED  survey drone 1 verifies north channel
[07]ACTION DISPATCHED  authorized command sent to survey_drone_1
[08]OBSERVATION RECEIVED north_channel is blocked
```

SIDE FILE: step06_verification_dispatch/S06_controller.py

SIDE FILE: step06_verification_dispatch/S06_flood_environment.py

Exit test

A route observation exists only after a committed verify_route action.

Step 6: Code: commitment before execution

A durable action is written with the exact record version that authorized it.

STEP 6 **Code: commitment before execution**

MISSION CONTROLLER

```python
chosen = self.planner.choose(
        [(plan_by_id[item["plan_id"]], item["decision"]) for item in assessed]
    )
    if chosen is None:
        raise RuntimeError("no admissible initial action")

    emit(
        "action_selected",
        f"Mission Controller selects: {chosen.first_action.short_label}.",
        plan_id=chosen.plan_id,
        action=chosen.first_action.model_dump(mode="json"),
    )
    chosen_record = record_by_plan[chosen.plan_id]
    first_commitment = self.runtime.commit(
        record=chosen_record,
        action=chosen.first_action,
    )
    emit(
        "action_dispatched",
        f"Authorized command sent to {chosen.first_action.actor_id}.",
        commitment_id=first_commitment.commitment_id,
        authorizing_record_id=chosen_record.record_id,
        authorizing_record_version=chosen_record.record_version,
    )
    first_outcome = self.environment.execute(chosen.first_action)
    if chosen.first_action.action_type == "verify_route":
        emit(
            "observation_received",
            f"{chosen.first_action.actor_id} reports "
            f"{first_outcome.observations['route_id']} is "
            f"{first_outcome.observations['route_status']}.",
            outcome_id=first_outcome.outcome_id,
            observations=first_outcome.observations,
        )
```

SIDE FILE: step06_verification_dispatch/S06_controller.py

TRACE RUNTIME COMMIT

```python
def commit(
    self,
    *,
    record: TraceRecord,
    action: ActionInstance,
) -> Commitment:
    commitment = Commitment(
        action=action,
        authorizing_record_id=record.record_id,
        authorizing_record_version=record.record_version,
        consumer_policy_version=self.policy.config.policy_version,
    )
    self.commitments.append(commitment, record)
    return commitment
```

The commitment object is the closure link: action ↔ authorizing_record_id/version ↔ consumer policy.

SIDE FILE: step06_verification_dispatch/S06_runtime.py

Step 7: Revise append-only and locally replan

Reality contradicts the north prediction; the old record remains and a new version rejects it.

STEP 7

Revise append-only and locally replan

Before

North record v2
status=DEFER
consumer=HOLD
repair=verify route

Observation

survey_drone_1
route_status=blocked
source and battery recorded

Revision

same record_id
record_version=3
status=REJECT
failed_gate=realized_contradiction

Replan

blocked north omitted
South candidate remains
verification preserved

Locality in the current code is route-specific candidate filtering. A general HTN dependency graph and arbitrary branch invalidation are future work.

SIDE FILE: step07_revision_repair/S07_controller.py

SIDE FILE: step07_revision_repair/S07_runtime.py

Exit test

Old record readable; revision supersedes it; north dispatch absent after re-observation.

Step 7: Code: append a contradiction instead of editing history

The revision adds outcome evidence, a new status, a failed gate, and a repair.

STEP 7 **Code: append a contradiction instead of editing history**

RUNTIME REVISION

```python
def revise_with_outcome(
    self,
    record: TraceRecord,
    evidence: WorldModelEvidence,
    *,
    new_status: TraceStatus,
    reason: str,
    repair: str,
) -> TraceRecord:
    evidence_ref = self.ledger.put(evidence)
    revised = record.model_copy(
        update={
            "record_version": record.record_version + 1,
            "evidence_refs": record.evidence_refs + (evidence_ref,),
            "final_status": new_status,
            "failed_gates": record.failed_gates + ("realized_contradiction",),
            "repair": repair,
            "supersedes_record_id": record.record_id,
            "supersedes_record_version": record.record_version,
            "metadata": {**record.metadata, "revision_reason": reason},
        }
    )
    self.repository.write(revised)
    return revised
```

CONTROLLER CONTRADICTION + REPLAN

```python
revisions: list[dict] = []
if (
    chosen.first_action.action_type == "verify_route"
    and first_outcome.observations.get("route_status") == "blocked"
):
    route_id = chosen.first_action.route_id
    dispatch_plan_id = (
        "north-direct"
        if route_id == "north_channel"
        else f"dispatch-{route_id}"
    )
    failed_record = record_by_plan[dispatch_plan_id]
    failed_plan = plan_by_id[dispatch_plan_id]
    failed_prediction = prediction_by_plan[dispatch_plan_id]
    failed_claim = self.probe.build_claim(failed_plan, failed_prediction)

    visual_payload = json.dumps(initial_observation, sort_keys=True).encode("utf-8")
    state = fuse_state(self.encoder.encode(visual_payload), initial_observation)
    realized_evidence = self._make_evidence(
        plan=failed_plan,
        prediction=failed_prediction,
        claim=failed_claim,
        state_hash=state.state_hash,
        observation_hash=sha256_value(initial_observation),
        outcome=first_outcome,
        contradicted=True,
    )
    revised = self.runtime.revise_with_outcome(
        failed_record,
        realized_evidence,
        new_status=TraceStatus.REJECT,
        reason=(
            f"{chosen.first_action.actor_id} observed that {route_id} is blocked."
        ),
        repair=(
            "Preserve the completed verification action and replan only "
            "the rescue-route branch that depended on the failed claim."
        ),
    )
    revisions.append(
        {
            "record_id": revised.record_id,
            "record_version": revised.record_version,
            "status": revised.final_status.value,
            "reason": revised.metadata["revision_reason"],
        }
    )
    emit(
        "trace_revised",
        "TRACE rejects the northern-route claim and preserves the "
        "completed verification action.",
        record_id=revised.record_id,
        record_version=revised.record_version,
    )

second_observation = self.environment.observe()
emit(
    "replanning",
    "Planner repairs only the route-dependent rescue branch.",
)
second_assessed, second_records, _, second_plans = self._assess_observation(
    second_observation,
    replanned=True,
)
```

No field in the prior committed record is changed in place.

The re-observed route report becomes blocked, so FloodPlanner.propose() no longer emits the north dispatch.

Step 8: Clear the final dispatch and record the rescue outcome

The final boat action is linked to a new TRACE record and the policy version that cleared it.

STEP 8 **Clear the final dispatch and record the rescue outcome**

Reassess
South support=0.93
OOD=0.08
CLEAR

Authority
IncidentCommander permits dispatch_rescue_boat

Commit
action + record ID/version written to commitment log

Execute
boat uses south_detour
people_reached=4

Persist
summary + timeline records + evidence + figures

```
[12] FINAL ACTION SELECTED  boat via South Detour
[13] FINAL ACTION DISPATCHED  Commander approval + TRACE clearance recorded
[14] RESCUE OUTCOME      success=True, people_reached=4
```

Faithfulness boundary: “Commander approval” is currently a deterministic role stub. It demonstrates the authority field and gate, not a human-in-the-loop interface.

SIDE FILE: step08_final_rescue/S08_controller.py

SIDE FILE: step08_final_rescue/S08_flood_environment.py

SIDE FILE: step08_final_rescue/S08_reporting.py

Detailed events 12-14

Step 8: Code: final authority, commitment, execution, and summary

The final record closure can be reconstructed from summary.json and the append-only logs.

STEP 8 Code: final authority, commitment, execution, and summary

```
emit(
    "final_action_selected",
    f"Mission Controller selects: {final_plan.first_action.short_label}.",
    plan_id=final_plan.plan_id,
    action=final_plan.first_action.model_dump(mode="json"),
  )
  final_record = second_records[final_plan.plan_id]
  final_commitment = self.runtime.commit(
    record=final_record,
    action=final_plan.first_action,
  )
  emit(
    "final_action_dispatched",
    f"Incident Commander approval and TRACE clearance authorize "
    f"{final_plan.first_action.actor_id}.",
    commitment_id=final_commitment.commitment_id,
    authorizing_record_id=final_record.record_id,
    authorizing_record_version=final_record.record_version,
  )
  final_outcome = self.environment.execute(final_plan.first_action)
  emit(
    "rescue_outcome",
    f"Rescue completed: success={final_outcome.success}, "
    f"people_reached={final_outcome.observations.get('people_reached', 0)}.",
    outcome_id=final_outcome.outcome_id,
    observations=final_outcome.observations,
  )

  summary = {
    "scenario": self.environment.scenario_id,
    "emergency_call": (
      active_call.model_dump(mode="json") if active_call is not None else None
    ),
    "entities": {
      "environment": "FloodEnvironment",
      "fleet": ["survey_drone_1", "rescue_boat_1"],
      "mission_controller": self.version,
      "incident_commander": self.incident_commander.role_id,
      "offline_scorer_in_control_loop": False,
    },
    "timeline": timeline,
    "initial_mission_controller_knowledge": initial_observation,
    "initial_controller_knowledge": initial_observation,
    "simulation_ground_truth_at_time_zero": initial_simulation_ground_truth,
    "simulation_ground_truth_after_run": self.environment.simulation_ground_truth(),
    "initial_assessments": assessed,
    "first_selected_plan": chosen.plan_id,
    "first_commitment_id": first_commitment.commitment_id,
    "first_commitment": first_commitment.model_dump(mode="json"),
    "first_action": chosen.first_action.model_dump(mode="json"),
    "first_outcome": first_outcome.model_dump(mode="json"),
    "revisions": revisions,
    "replanned_mission_controller_knowledge": second_observation,
    "replanned_assessments": second_assessed,
    "final_selected_plan": final_plan.plan_id,
    "final_commitment_id": final_commitment.commitment_id,
    "final_commitment": final_commitment.model_dump(mode="json"),
    "final_action": final_plan.first_action.model_dump(mode="json"),
    "final_outcome": final_outcome.model_dump(mode="json"),
    "rescued": self.environment.rescued,
    "record_chain_valid": self.runtime.repository.verify_chain(),
  }

  if output is not None:
    output = Path(output)
    output.mkdir(parents=True, exist_ok=True)
    write_json(output / "summary.json", summary)
```

- Select supported south plan.
- Create final commitment.
- Log record ID and version.
- Execute simulated boat action.
- Write final outcome and summary.
- Verify record hash chain.

Exit test

rescued = true
people_reached = 4
record_chain_valid = true

Authoritative source: MissionController.run_episode

The detailed fourteen-event log maps cleanly to the eight steps

The CLI stays granular for audit; the deck groups events by architectural responsibility.

STEP 1	**01 CALL RECEIVED**	intake.py / emergency_cli.py
STEP 2	**02 MISSION STATE**	flood_env.py
STEP 3	**03-05 PLAN ASSESSED**	planner.py
STEP 4	**03-05 prediction + claim inside assessment**	toy.py / probes.py
STEP 5	**03-05 TRACE status and decision**	policy.py / runtime.py
STEP 6	**06-08 select / dispatch / observe**	controller.py / flood_env.py
STEP 7	**09-11 revise / replan / reassess**	runtime.py / controller.py / planner.py
STEP 8	**12-14 select / dispatch / outcome**	controller.py / reporting.py

The event log is an operational trace, not the TRACE record itself. TRACE records are stored separately under records/trace.jsonl.

The episode as it actually runs

This output was reproduced from the packaged code with PYTHONPATH=src.

```
[01] CALL_RECEIVED    Emergency call reports 4 people at Riverside Apartments.
[02] MISSION_STATE    Mission Controller grounds the incident at Riverside Apartments; North Channel is unknown.
[03] PLAN_ASSESSED    Dispatch rescue_boat_1 from rescue_base to riverside_apartments via North Channel -> HOLD (support=0.28, OOD=0.82).
[04] PLAN_ASSESSED    Send survey_drone_1 to verify North Channel -> CLEAR (support=0.97, OOD=0.04).
[05] PLAN_ASSESSED    Dispatch rescue_boat_1 from rescue_base to riverside_apartments via South Detour -> CLEAR (support=0.93, OOD=0.08).
[06] ACTION_SELECTED  Mission Controller selects: survey drone 1 verifies north channel.
[07] ACTION_DISPATCHED Authorized command sent to survey_drone_1.
[08] OBSERVATION_RECEIVED survey_drone_1 reports north_channel is blocked.
[09] TRACE_REVISED    TRACE rejects the northern-route claim and preserves the completed verification action.
[10] REPLANNING       Planner repairs only the route-dependent rescue branch.
[11] PLAN_REASSESSED  Dispatch rescue_boat_1 from rescue_base to riverside_apartments via South Detour -> CLEAR.
[12] FINAL_ACTION_SELECTED Mission Controller selects: rescue boat 1 dispatches to riverside apartments via south detour.
[13] FINAL_ACTION_DISPATCHED Incident Commander approval and TRACE clearance authorize rescue_boat_1.
[14] RESCUE_OUTCOME   Rescue completed: success=True, people_reached=4.
```

SIDE FILE: sample_run/timeline.txt

No “forthcoming” placeholder on this slide

What one run leaves behind

Every important transition has a durable artifact.

```
artifacts/runs/call_001/
├── emergency_call.json
├── timeline.txt
├── summary.json
├── records/trace.jsonl
├── evidence/*.json
├── commitments/commitments.jsonl
└── figures/
    ├── operational_cast.svg
    ├── mission_controller_knowledge.svg
    ├── simulation_ground_truth.svg
    └── action_timeline.svg
```

Call

Original report + normalized location/count/deadline

Evidence

Versioned model outputs, hashes, assumptions, support

Records

Status, gates, missing items, repair, consumer action

Commitments

Exact action + authorizing record ID/version

Outcomes

What the simulator revealed after execution

Closure can be checked from stored files alone

What is implemented, what is a fixture, and what remains future work

A faithful tutorial must not let a clean demo masquerade as a completed research system.

Component	Current status	Boundary
Emergency-call parser	**Implemented**	Closed alias registry + deterministic count extraction
Flood environment	**Implemented teaching simulator**	Static scenario; no dynamic hydrology
Planner	**Implemented teaching planner**	Three candidate patterns; no HTN/MRTA solver
Predictor	**Deterministic fixture**	No JEPA or learned flood dynamics yet
Claim bridge	**Implemented**	Predictive claims only in this demo
TRACE policy/runtime	**Implemented**	Domain-specific gates; no full debate/attack graph
Authority	**Teaching stub**	No live human approval interface
Dispatcher	**Simulated**	No physical drone/boat integration
Local repair	**Implemented narrowly**	Route-specific filtering, not general dependency repair

This is an executable architectural demonstration. It is not yet the AAAI evaluation, a real rescue system, or evidence that JEPA improves flood response.

Verification matrix

Each step has an observable exit condition and at least one relevant test or artifact.

Step	Test / artifact	Pass condition
1 Call	test_emergency_call.py	Call grounds location/count; ambiguity fails explicitly
2 State	test_entity_model.py	Controller view excludes hidden route truth
3 Plans	test_end_to_end.py	Grounded actions and candidate set
4 Prediction	test_end_to_end.py	Separate success/support/OOD/uncertainty
5 Gate	test_policy.py	Unsupported high-confidence dispatch is held
6 Verify	test_end_to_end.py	Verification has commitment and returns route observation
7 Revise	test_end_to_end.py	Old record preserved; revision and south repair
8 Rescue	test_closure.py + end-to-end	Final commitment cites record; chain valid; four reached

```
pytest
```

Expected: core suite passes; optional PyTorch/V-JEPA adapter test may skip before ML installation.

Code pack and file labels

Long source files are supplied exactly, grouped by the step that uses them.

```
docs/eight_step_deck/
├── side_files/
│   ├── CODE_MAP.md
│   ├── RUN_EIGHT_STEPS.md
│   ├── step01_call_intake/
│   ├── step02_mission_state/
│   ├── step03_planning/
│   ├── step04_prediction_claims/
│   ├── step05_trace_gate/
│   ├── step06_verification_dispatch/
│   ├── step07_revision_repair/
│   ├── step08_final_rescue/
│   ├── complete_source/
│   └── sample_run/
├── build_eight_step_deck.js
├── TRACE_JEPA_Flood_SAR_8_Steps.pptx
└── TRACE_JEPA_Flood_SAR_8_Steps.pdf
```

- Every S01-S08 file is an exact source copy.
- CODE_MAP.md maps step → authoritative path → functions.
- The deck shows short excerpts only.
- complete_source/ mirrors package, configs, and tests.
- sample_run/ contains the verified episode artifacts.

Use in class

Students first read the step, run the command, inspect the artifact, then open the side file when the excerpt is insufficient.

SIDE FILE:CODE_MAP.md

No code is reachable only by hunting through slides

What comes next after the eight-step baseline

Replace fixtures one seam at a time without moving the TRACE boundary.

A	**Real geocoder**	Replace closed aliases; retain EmergencyCall contract
B	**Real visual encoder**	Replace MockVideoEncoder with V-JEPA 2.1 feature cache
C	**Learned dynamics**	Replace ToyActionPrefixPredictor with flood-domain predictor
D	**Calibrated probes**	Separate plan success from predicate probability
E	**General planner**	HTN/MRTA + explicit dependency graph
F	**Human interface**	Signed Incident Commander approval workflow
G	**Evaluation**	Fixed scenarios; vary audit substrate; report safety/utility/latency

The baseline is valuable because every replacement can be checked against the same eight-step contract and durable artifacts.

Definition of done

A reviewer can answer all eight questions from stored artifacts alone.

1. What did the caller report, and how was the location grounded?
2. What did Mission Control know, and what remained hidden?
3. Which candidate actions were structurally available?
4. What did the predictor claim, with what support and uncertainty?
5. Which TRACE gates passed or failed, and what repair was named?
6. Which evidence-seeking action was authorized and executed?
7. What observation forced revision, and which branch changed?
8. Which record and policy authorized the final rescue, and what happened?

Better models may replace the fixtures later. The accountability boundary and these reconstruction questions stay fixed.